



Presigned URLs

Presigned URLs are an [S3 concept](#) for sharing direct access to your bucket without revealing your token secret. A presigned URL authorizes anyone with the URL to perform an action to the S3 compatibility endpoint for an R2 bucket. By default, the S3 endpoint requires an `AUTHORIZATION` header signed by your token. Every presigned URL has S3 parameters and search parameters containing the signature information that would be present in an `AUTHORIZATION` header. The performable action is restricted to a specific resource, an [operation](#), and has an associated timeout.

There are three kinds of resources in R2:

1. **Account:** For account-level operations (such as `CreateBucket`, `ListBuckets`, `DeleteBucket`) the identifier is the account ID.
2. **Bucket:** For bucket-level operations (such as `ListObjects`, `PutBucketCors`) the identifier is the account ID, and bucket name.
3. **Object:** For object-level operations (such as `GetObject`, `PutObject`, `CreateMultipartUpload`) the identifier is the account ID, bucket name, and object path.

All parts of the identifier are part of the presigned URL.

You cannot change the resource being accessed after the request is signed. For example, trying to change the bucket name to access the same object in a different bucket will return a 403 with an error code of `SignatureDoesNotMatch`.

Presigned URLs must have a defined expiry. You can set a timeout from one second to 7 days (604,800 seconds) into the future. The URL will contain the time

when the URL was generated (X-Amz-Date) and the timeout (X-Amz-Expires) as search parameters. These search parameters are signed and tampering with them will result in 403 with an error code of `SignatureDoesNotMatch`.

Presigned URLs are generated with no communication with R2 and must be generated by an application with access to your R2 bucket's credentials.

Presigned URL use cases

There are three ways to grant an application access to R2:

1. The application has its own copy of an [R2 API token](#).
2. The application requests a copy of an R2 API token from a vault application and promises to not permanently store that token locally.
3. The application requests a central application to give it a presigned URL it can use to perform an

action.

In scenarios 1 and 2, if the application or vault application is compromised, the holder of the token can perform arbitrary actions.

Scenario 3 keeps the credential secret. If the application making a presigned URL request to the central application leaks that URL, but the central application does not have its key storage system compromised, the impact is limited to one operation on the specific resource that was signed.

Additionally, the central application can perform monitoring, auditing, logging tasks so you can review when a request was made to perform an operation on a specific resource. In the event of a security incident, you can use a central application's logging functionality to review details of the incident.

The central application can also perform policy enforcement. For example, if you have an application responsible for uploading resources, you can restrict

the upload to a specific bucket or folder within a bucket. The requesting application can obtain a JSON Web Token (JWT) from your authorization service to sign a request to the central application. The central application then uses the information contained in the JWT to validate the inbound request parameters.

The central application can be, for example, a Cloudflare Worker. Worker secrets are cryptographically impossible to obtain outside of your script running on the Workers runtime. If you do not store a copy of the secret elsewhere and do not have your code log the secret somewhere, your Worker secret will remain secure. However, as previously mentioned, presigned URLs are generated outside of R2 and all that's required is the secret + an implementation of the signing algorithm, so you can generate them anywhere.

Another potential use case for presigned URLs is debugging. For example, if you are debugging your application and want to grant temporary access to a

specific test object in a production environment, you can do this without needing to share the underlying token and remembering to revoke it.

Supported HTTP methods

R2 currently supports the following methods when generating a presigned URL:

- GET: allows a user to fetch an object from a bucket
- HEAD: allows a user to fetch an object's metadata from a bucket
- PUT: allows a user to upload an object to a bucket
- DELETE: allows a user to delete an object from a bucket

POST, which performs uploads via native HTML forms, is not currently supported.

Presigned URL alternative with Workers

A valid alternative design to presigned URLs is to use a Worker with a [binding](#) that implements your security policy.

Bindings

A binding is how your Worker interacts with external resources such as [KV Namespaces](#), [Durable Objects](#), or [R2 Buckets](#). A binding is a runtime variable that the Workers runtime provides to your code. You can declare a variable name in your Wrangler file that will be bound to these resources at runtime, and interact with them through this variable. Every binding's variable name and behavior is determined by you when deploying the Worker. Refer to [Environment Variables](#) for more information.

A binding is defined in the Wrangler file of your Worker project's directory.

Refer to [Use R2 from Workers](#) to learn how to bind a bucket to a Worker and use the binding to interact with your bucket.

Generate presigned URLs

Generate a presigned URL by referring to the following examples:

- [AWS SDK for Go](#)
- [AWS SDK for JS v3](#)
- [AWS SDK for JS](#)
- [AWS SDK for PHP](#)
- [AWS CLI](#)

Example of generating presigned URLs

A possible use case may be restricting an application to only be able to upload to a specific URL. With presigned URLs, your central signing application might look like the following JavaScript code running on Cloudflare Workers, workerd, or another platform (you might have to update the code based on the platform you are using).

If the application received a request for `https://example.com/uploads/dog.png`, it would respond with a presigned URL allowing a user to upload to your R2 bucket at the `/uploads/dog.png` path.

To create a presigned URL, you will need to either use a package that implements the signing algorithm, or implement the signing algorithm yourself. In this example, the `aws4fetch` package is used. You also need to have an access key ID and a secret access key. Refer to [R2 API tokens](#) for more information.

TypeScript

```
import { AwsClient } from "aws4fetch";

// Create a new client
// Replace with your own access key ID and secret
// Make sure to store these securely and not expose them
const client = new AwsClient({
  accessKeyId: "",
  secretAccessKey: "",
});

export default {
  async fetch(req): Promise<Response> {
    // This is just an example to demonstrating using
    aws4fetch to generate a presigned URL.
    // This Worker should not be used as-is as it does not
    authenticate the request, meaning
    // that anyone can upload to your bucket.
    //
    // Consider implementing authorization, such as using a
    preshared secret in a request header.
    const requestPath = new URL(req.url).pathname

    // Cannot upload to the root of a bucket
    if (requestPath === "/") {
```

```
    return new Response("Missing a filepath", {
  });
}

// Replace with your bucket name and account
const bucketName = "";
const accountId = "";

const url = new URL(
`https://${bucketName}.${accountId}.r2.cloudflare
`);

// preserve the original path
url.pathname = requestPath;

// Specify a custom expiry for the presigned
seconds
url.searchParams.set("X-Amz-Expires", "3600")

const signed = await client.sign(
  new Request(url, {
    method: "PUT",
  }),
  {
```

```
        aws: { signQuery: true },
    },
);

// Caller can now use this URL to upload to the bucket
return new Response(signed.url, { status: 200 });

// ... handle other kinds of requests
} satisfies ExportedHandler;
```

Differences between presigned URLs and R2 binding

- When using an R2 binding, you will not need any token secrets in your Worker code. Instead, in your [Wrangler configuration file](#), you will create a [binding](#) to your R2 bucket. Additionally, authorization is handled in-line, which can reduce latency.

- When using presigned URLs, you will need to create and use the token secrets in your Worker code.

In some cases, R2 bindings let you implement certain functionality more easily. For example, if you wanted to offer a write-once guarantee so that users can only upload to a path once:

- With R2 binding: You only need to pass the header once.
- With presigned URLs: You need to first sign specific headers, then request the user to send the same headers.

R2 binding example

Presigned URL example

If you are using R2 bindings, you would change your upload to:

TypeScript

```
const existingObject = await
env.R2_BUCKET.put(key, request.body, {
  onlyIf: {
    // No objects will have been uploaded
    before September 28th, 2021 which
    // is the initial R2 announcement.
    uploadedBefore: new Date(1632844800000),
  },
});
if (existingObject?.etag !==
request.headers.get("etag")) {
  return new Response("attempt to overwrite
object", { status: 400 });
}
```

When using R2 bindings, you may need to consider the following limitations:

- You cannot upload more than 100 MiB (200 MiB for Business customers) when using R2 bindings.
- Enterprise customers can upload 500 MiB by default and can ask their account team to raise this limit.
- Detecting [precondition failures](#) is currently easier with presigned URLs as compared with R2 bindings.

Note that these limitations depend on R2's extension for conditional uploads. Amazon's S3 service does not offer such functionality at this time.

Differences between presigned URLs and public buckets

Presigned URLs share some superficial similarity with public buckets. If you give out presigned URLs only for GET/HEAD operations on specific objects in a bucket, then your presigned URL functionality is mostly similar

to public buckets. The notable exception is that any custom metadata associated with the object is rendered in headers with the `x-amz-meta-` prefix. Any error responses are returned as XML documents, as they would with normal non-presigned S3 access.

Presigned URLs can be generated for any S3 operation. After a presigned URL is generated it can be reused as many times as the holder of the URL wants until the signed expiry date.

[Public buckets](#) are available on a regular HTTP endpoint. By default, there is no authorization or access controls associated with a public bucket. Anyone with a public bucket URL can access an object in that public bucket. If you are using a custom domain to expose the R2 bucket, you can manage authorization and access controls as you would for a Cloudflare zone. Public buckets only provide GET/HEAD on a known object path. Public bucket errors are rendered as HTML pages.

Choosing between presigned URLs and public buckets is dependent on your specific use case. You can also use both if your architecture should use public buckets in one situation and presigned URLs in another. It is useful to note that presigned URLs will expose your account ID and bucket name to whoever gets a copy of the URL. Public bucket URLs do not contain the account ID or bucket name. Typically, you will not share presigned URLs directly with end users or browsers, as presigned URLs are used more for internal applications.

Limitations

Presigned URLs can only be used with the `<accountid>.r2.cloudflarestorage.com` S3 API domain and cannot be used with custom domains. Instead, you can use the [general purpose HMAC validation feature of the WAF](#), which requires a Pro plan or above.

Related resources

- [Create a public bucket](#)
- [Storing user generated content](#)

Was this helpful?



Previous

← Extensions

Next

Use R2 from Workers →

Last updated: May 12, 2025

Resources

API

New to Cloudflare?

Directory

Sponsorships

Open Source

Support

Help Center

System Status

Compliance

GDPR

Company

[cloudflare.com](#)

[Our team](#)

[Careers](#)

Tools

[Cloudflare Radar](#)

[Speed Test](#)

[Is BGP Safe Yet?](#)

[RPKI Toolkit](#)

[Certificate Transparency](#)

Community

 [X](#)

 [Discord](#)

 [YouTube](#)

 [GitHub](#)

© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) •  [Cookie Preferences](#)